Plano de Curso

Design and Verification Track

Documento de Planejamento

Gabriel Villanova Novaes Magalhães
Instrutor

Campina Grande, PB

Fevereiro de 2026

1 Resumo

O **Design and Verification Track** é um programa voltado à formação completa de engenheiros de verificação de hardware digital com foco na indústria de semicondutores. O curso é baseado na trilha Purple e contempla tanto os fundamentos da microeletrônica quanto o desenvolvimento de ambientes avançados de verificação utilizando SystemVerilog, UVM e ferramentas profissionais da Synopsys.

Na etapa inicial, são abordados os pré-requisitos técnicos essenciais, incluindo fundamentos de CMOS, VLSI, Very Deep Submicron (VDSM), projeto digital e fluxo de desenvolvimento de ASIC. Essa base fornece ao aluno a compreensão tecnológica necessária para contextualizar as atividades de verificação dentro do processo completo de desenvolvimento de circuitos integrados.

Em seguida, o curso aprofunda o uso do SystemVerilog como linguagem de verificação, explorando recursos como classes, randomização, cobertura funcional e arquitetura de testbenches. A trilha inclui o desenvolvimento prático de ambientes de verificação baseados em UVM (Universal Verification Methodology), além do estudo de SystemVerilog Assertions (SVA) para verificação formal, incluindo sua aplicação em ferramentas como VCS, Verdi e VC Formal.

Ao final do programa, o participante estará apto a projetar, implementar e depurar ambientes profissionais de verificação e compreender o papel estratégico da verificação no fluxo moderno de desenvolvimento de ASIC.

2 Objetivos

Objetivo Geral

Formar engenheiros capazes de atuar profissionalmente na área de Verificação Funcional de projetos ASIC, compreendendo o fluxo completo de desenvolvimento de circuitos integrados — desde os fundamentos tecnológicos até a validação funcional — e dominando o uso de SystemVerilog, UVM e ferramentas industriais de simulação, debug e verificação formal.

Objetivos Específicos

Ao final do curso, o participante deverá ser capaz de:

- Compreender os fundamentos tecnológicos de CMOS, VLSI e tecnologias VDSM aplicados ao desenvolvimento de ASIC.
- Entender o fluxo de projeto de circuitos integrados, incluindo etapas de especificação, modelagem, verificação e integração.
- Modelar e simular projetos digitais utilizando Verilog e SystemVerilog.
- Desenvolver testbenches estruturados e escaláveis, aplicando boas práticas de arquitetura de verificação.
- Utilizar recursos avançados do SystemVerilog, como classes, randomização, constraints, coverage e interfaces.
- Escrever e integrar SystemVerilog Assertions (SVA) para verificação dinâmica baseada em simulação.
- Aplicar conceitos de verificação formal utilizando propriedades e ferramentas dedicadas.
- Construir ambientes completos baseados em UVM, compreendendo componentes, fases, comunicação TLM e mecanismos de configuração.
- Implementar estratégias de verificação orientadas a cobertura funcional e estrutural.
- Utilizar ferramentas profissionais como VCS e Verdi para simulação, debug e análise de cobertura.
- Identificar, analisar e depurar falhas funcionais em dispositivos sob teste (DUTs).

3 Conteúdo / Ementa

A trilha é estruturada em duas fases complementares distribuídas em 20 semanas. A Fase 1 contempla os fundamentos tecnológicos e o fluxo de desenvolvimento de circuitos integrados. A Fase 2 é dedicada à Verificação Funcional, com foco em SystemVerilog, UVM, Assertions e ferramentas industriais.

Fase 1 – Fundamentos Tecnológicos e de Projeto (Semanas 1–4)

1. Pré-requisitos para Ambiente de Projeto VLSI

Ambiente Linux para desenvolvimento; execução de ferramentas EDA em modo interativo e batch; fundamentos de automação de fluxo utilizando scripts; introdução a Tcl/Tk, Python e Perl aplicados à manipulação de arquivos, relatórios e processos de projeto.

2. Fundamentos de CMOS

Física básica de semicondutores; materiais semicondutores; estrutura e operação de dispositivos MOS; tecnologia de fabricação CMOS; regras de layout; modelos SPICE; variações de processo e corners; circuitos CMOS básicos; células padrão; circuitos sequenciais; memórias on-chip (SRAM e DRAM); fundamentos de caracterização.

3. Fundamentos de VLSI

Representação de circuitos em diagramas e modelos HDL; bibliotecas tecnológicas e células padrão; regras de projeto; síntese lógica; restrições de projeto; modelos de atraso; métricas de potência, desempenho e área; introdução a subsistemas e implementação em FPGA.

4. Very Deep Submicron (VDSM)

Tendências tecnológicas em nós avançados; desafios de temporização; desafios de interconexão; consumo de potência e técnicas de baixo consumo; clock e crossing de domínios (CDC); variações de processo; confiabilidade (eletromigração, latch-up, efeito antena); células auxiliares e considerações físicas.

5. Fundamentos de Projeto Digital

Sistemas numéricos; álgebra booleana; técnicas de otimização lógica; circuitos combinacionais; circuitos sequenciais; máquinas de estados finitos (FSM); fundamentos de clock; temporização e verificação de timing.

6. Fluxo de Projeto ASIC

Tipos de circuitos integrados; linguagens de descrição de sistemas; modelagem e representação de projeto em VHDL e SystemVerilog; verificação funcional no fluxo VLSI; síntese lógica; design-for-test (DFT); fluxo de implementação física; verificação física e signoff.

Fase 2 – Verificação Funcional (Semanas 5–20)

Bloco 1 – Linguagens HDL, Síntese e Fundamentos de Verificação (Semanas 5–8)

1. Verilog (Refresher)

Modelagem RTL utilizando Verilog; estruturas de descrição de hardware; diferenças entre compilação e síntese; construções para síntese; construções para verificação; modelagem de testbenches; simulação de projetos de referência.

2. VHDL (Refresher)

Modelagem RTL utilizando VHDL; estruturas e construções da linguagem; distinção entre síntese e compilação; VHDL para síntese; VHDL para verificação; modelagem de testbenches; simulação de projetos de referência.

3. SystemVerilog (Refresher)

Modelagem RTL em SystemVerilog; extensões da linguagem para síntese e verificação; comparação entre construções de síntese e de simulação; modelagem de testbenches; simulação e validação de projetos de referência.

4. Síntese de Projeto RTL

Conceitos de síntese lógica; arquivos de projeto e restrições; geração de netlists; técnicas de otimização de tempo, área e potência; noções de high-level synthesis; fluxo básico de síntese no contexto ASIC.

5. Fundamentos de Verificação de Projeto

Conceitos de verificação funcional; metodologias de verificação em VLSI; linguagens de verificação de hardware; estrutura de testbenches; introdução a estímulo randômico e verificação orientada a cobertura; visão geral de ferramentas industriais de simulação e debug.

6. Verificação Abrangente

Componentes de testbench; geração de estímulo; técnicas de checagem de respostas; coleta e interpretação de cobertura funcional; análise de relatórios de cobertura.

7. Linguagem SystemVerilog para Testbench

Ambiente de verificação em SystemVerilog; programação orientada a objetos aplicada à verificação; geração de estímulo constrained-random; drivers e monitores; comunicação concorrente (eventos, semáforos, mailboxes); pacotes; cobertura funcional; introdução à estrutura metodológica baseada em classes.

Bloco 2 – SystemVerilog Assertions e Verificação Formal (Semanas 9–12)

1. SystemVerilog Assertions (SVA)

Conceitos e fundamentos da linguagem SVA; assertions imediatas e concorrentes; blocos de ação; controle, desabilitação e combinação de assertions; sequences e operadores temporais; reutilização e escalabilidade de propriedades; embedding e binding de módulos de assertions; bibliotecas de assertions; cobertura de assertions e cover properties; análise e depuração de falhas utilizando ferramentas de simulação e debug.

2. SystemVerilog Assertions para Verificação Formal

Visão geral da linguagem SVA aplicada à verificação formal; escrita de propriedades para ferramentas formais; estrutura de formal testbench; boas práticas para modelagem de propriedades eficientes; uso de assertions como entrada para *engines* formais; integração entre verificação dinâmica e formal.

Bloco 3 – Verificação com UVM (Semanas 13–20)

O bloco aborda o desenvolvimento completo de ambientes de verificação baseados na metodologia UVM e no curso **SystemVerilog Verification using UVM**, contemplando:

- **Arquitetura e Estrutura UVM:** organização hierárquica de componentes (test, env, agent, driver, monitor, sequencer); modelagem de transações; separação entre geração de estímulo e aplicação ao DUT; agentes ativos e passivos reutilizáveis.
- **Comunicação e Integração:** uso de TLM ports, exports e analysis ports; integração de scoreboards; coleta de cobertura funcional; sincronização entre componentes.
- **Controle e Configuração:** geração de estímulo constrained-random; gerenciamento de testes orientados a cenários; uso da configuration database; substituição e extensão de componentes via factory; aplicação de callbacks.
- **Execução e Gerenciamento de Simulação:** mecanismos de phasing e objections; sistema de mensagens e relatórios; coordenação de sequences complexas e sequencers hierárquicos.
- **Register Abstraction Layer (RAL):** modelagem e abstração de registradores; espelhamento e verificação de registradores; integração de RAL com sequences e cobertura funcional.
- **Aplicação Prática:** desenvolvimento progressivo de um ambiente completo de verificação baseado em UVM, incluindo extensão, reuso e manutenção de casos de teste.

4 Metodologia de Execução

O curso será conduzido de forma teórico-prática, combinando exposição de conteúdo com aplicação direta em exercícios e ferramentas da Synopsys.

As aulas serão estruturadas da seguinte forma:

- **aulas expositivas com apoio de slides;**
- **explicações complementares em quadro para detalhamento de conceitos;**
- **demonstrações ao vivo utilizando ambiente Linux e ferramentas;**
- **desenvolvimento prático de códigos RTL, assertions e ambientes UVM;**
- **análise de simulações, formas de onda e relatórios de cobertura**.

Ao longo do curso, os participantes realizarão atividades práticas para consolidar os conteúdos abordados, incluindo exercícios de modelagem, verificação e depuração.

Na Fase 2, especialmente no bloco de UVM, será desenvolvido progressivamente um ambiente completo de verificação, permitindo ao participante aplicar os conceitos estudados em um contexto próximo ao ambiente industrial.

A metodologia prioriza a compreensão dos fundamentos, a prática contínua e a evolução técnica gradual do participante.

5 Acompanhamento / Avaliação

O acompanhamento do participante será realizado de forma contínua ao longo das 20 semanas, combinando avaliações teóricas e práticas alinhadas ao conteúdo de cada fase e bloco. As avaliações abordam os principais tópicos expostos no curso, incluindo atividades práticas, exercícios de laboratório e interpretação de resultados obtidos com ferramentas de simulação, debug e verificação.

Instrumentos de Avaliação

- **Provas por etapa (5 avaliações):** serão realizadas cinco avaliações ao longo da trilha. As duas primeiras ocorrem durante a Fase 1 e as demais ao final de cada bloco da Fase 2.
- **Códigos de laboratório:** cada aluno deverá entregar o material individual contendo os códigos desenvolvidos nas atividades práticas. Cada projeto ou repositório de laboratório deverá incluir um arquivo `README.md` descrevendo a implementação realizada e os passos necessários para reproduzir os resultados. O material deverá ser submetido no formato `.tar.gz` ao final de cada avaliação.

Estrutura das Provas

- **Fase 1:**
  - **P1 (Semanas 1–2):** pré-requisitos e fundamentos iniciais (ambiente, automação, conceitos tecnológicos).
  - **P2 (Semanas 3–4):** fundamentos de projeto digital e fluxo de desenvolvimento ASIC.
- **Fase 2:**
  - **P3 (final do Bloco 1 – Semanas 5–8):** linguagens HDL, síntese RTL e fundamentos de verificação.
  - **P4 (final do Bloco 2 – Semanas 9–12):** SystemVerilog Assertions e fundamentos de verificação formal.
  - **P5 (final do Bloco 3 – Semanas 13–20):** desenvolvimento de ambiente UVM completo, incluindo configuração, integração, cobertura e práticas de reuso.

As provas poderão incluir questões objetivas e discursivas, interpretação de formas de onda e relatórios, elaboração de trechos de código (SystemVerilog/SVA/UVM) e execução de tarefas práticas, de acordo com o conteúdo da etapa.

Pesos e Cálculo da Nota Final

A nota final (NF) será calculada por média ponderada, priorizando o desempenho nas avaliações teóricas/práticas individuais, com os códigos compondo uma parcela complementar da nota. De modo geral, adota-se a proporção de 85% para prova e 15% para códigos.

- **P1 (Fase 1 – Semanas 1–2):**

$$P1 = 0.85 \times \text{Prova P1} + 0.15 \times C_{12}$$

  Peso na média final: 10%

- **P2 (Fase 1 – Semanas 3–4):**

$$P2 = 0.85 \times \text{Prova P2} + 0.15 \times C_{12}$$

  Peso na média final: 10%

- **P3 (Fase 2 – Bloco 1):**

$$P3 = 0.85 \times \text{Prova} + 0.15 \times \text{Códigos}$$

  Peso na média final: 20%

- **P4 (Fase 2 – Bloco 2):**

$$P4 = 0.85 \times \text{Prova} + 0.15 \times \text{Códigos}$$

  Peso na média final: 20%

- **P5 (Fase 2 – Bloco 3 / UVM):**

$$P5 = 0.85 \times \text{Prova} + 0.15 \times \text{Códigos}$$

  Peso na média final: 40%

Onde C_{12} representa a média das notas de códigos da Fase 1:

$$C_{12} = \frac{\text{Códigos P1} + \text{Códigos P2}}{2}$$

Assim, a nota final é calculada como:

$$NF = 0.10P1 + 0.10P2 + 0.20P3 + 0.20P4 + 0.40P5$$

6 Cronograma

Timeline (20 semanas)

Tópicos	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Fase 1 – Fundamentos																				
S1: Linux + scripts + EDA																				
S2: CMOS																				
S3: VLSI + VDSM																				
S4: Digital + ASIC Flow																				
Fase 2 – Verificação Funcional																				
Bloco 1 – HDL + Síntese + Verif.																				
S5: Verilog/VHDL																				
S6: SystemVerilog																				
S7: Síntese RTL + metodologias																				
S8: TB + CRV + coverage																				
Bloco 2 – Assertions + Formal																				
S9: SVA (imm./conc.)																				
S10: seq. + libs + cov																				
S11: Formal TB + props																				
S12: boas práticas + integração																				
Bloco 3 – UVM																				
S13: overview + transactions																				
S14: agents + TLM																				
S15: config_db + factory																				
S16: scoreboard + coverage																				
S17: phasing + objections + report																				
S18: seq. avançadas + callbacks																				
S19: RAL + integração																				
S20: consolidação do ambiente																				

Referências

[1] L. Scheffer, L. Lavagno e G. Martin, ed., *EDA for IC Implementation, Circuit Design, and Process Technology*. CRC Press, 2006.

[2] N. H. E. Weste e D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4ª ed. Addison-Wesley, 2010.

[3] K. A. Meade e S. Rosenberg, *A Practical Guide to Adopting the Universal Verification Methodology (UVM)*. Wiley, 2013.

[4] R. Salemi, *The UVM Primer: An Introduction to the Universal Verification Methodology*. Boston Light Press, 2013.

[5] Siemens Verification Academy, *UVM Cookbook*, `https://verificationacademy.com/cookbook/uvm`, Acesso em 2026, 2023.

[6] P. James, *Verification Plans: The Five-Day Verification Strategy for Modern Hardware Verification Languages*. Springer, 2003.

[7] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE, 2017.

[8] B. Cohen, *SystemVerilog Assertions Handbook*, 4ª ed. SystemVerilog.us, 2016.

[9] *UVM 1.2 Class Reference*, Available at: https://accellera.org, Accellera Systems Initiative, 2015.